Data Resources. We access the dataset at `https://research.facebook.com/downloads/babi/`.The dataset is made available through our benchmark. The dataset has a datasheet at `https://huggingface.co/datasets/babi_qa`. Other resources on the dataset include (Kumar et al., 2016), (Weston, 2016) and (Dehghani et al., 2018).

E.3.4 Dyck Languages

Scenario Description. As we mentioned in §5.3.1: REASONING-PRIMITIVES, we designed this task to test the ability of language models to process and encode hierarchical information. Recent studies (Sennhauser & Berwick, 2018; Hao et al., 2018; Skachkova et al., 2018; Suzgun et al., 2019a;b; Hahn, 2020; Bhattamishra et al., 2020; Hewitt et al., 2020; Ebrahimi et al., 2020; Merrill, 2021, *inter alia*) have considered different variants of Dyck languages to investigate to what extend language models can emulate (deterministic) push-down automata. For our intents and purposes, we formulated this task as a conditional-generation task, wherein we predict the sequence of closing parentheses of a $\mathcal{D}_3$ word without its last few closing parenthesis.

Data. We generated the data in the style of Suzgun et al. (2019b). The input-output pairs are similar to those found in BIG-Bench (Srivastava et al., 2022) for $\mathcal{D}_4$. No additional pre-processing or data-augmentation was done.

E.3.5 GSM8K

Scenario Description. **GSM8K** (Cobbe et al., 2021) is a mathematical reasoning scenario that features grade-school-level math word problems. For example, one input-output pair in the train set is: "Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did Natalia sell altogether in April and May?" ⇒ "Natalia sold 48/2 = «48/2=24»24 clips in May. Natalia sold 48+24 = «48+24=72»72 clips altogether in April and May. The answer is 72".

In general, for this scenario, inputs will be math word problems consisting of several sentences and outputs will be step-by-step solutions to those problems requiring 2-8 calculations in the standard solution. We format the prompts similarly to Wei et al. (2022c). Although some implementations add a calculator for the text between "«" and "»' which automatically evaluates the computation for the language model (e.g. "«2+2=5»" becomes "«2+2=4»"), we allow the models to solve the arithmetic problems themselves and measure accuracy based on exact match.

Data. Cobbe et al. (2021) created the dataset by asking workers contracted through Surge AI to come up with math word problems, and then asked other contractors to solve those problems. If the proposed answer and derived answer matched, then the problem would be added to the dataset. The train-test splits for the dataset are 7500-1000 problems.

Pre-processing. No special pre-processing is performed.

Data Resources. We access the dataset at `https://github.com/openai/grade-school-math`. Although a datasheet is not provided by the authors, the dataset has a partial datasheet available at `https://huggingface.co/datasets/gsm8k`.

E.3.6 MATH

Scenario Description. **MATH** (Hendrycks et al., 2021d) is a dataset of real mathematics contest problems expressed in natural language and LaTeX(using Asymptote for figures). Solutions follow the same format and answers are boxed numbers (possibly real, decimal, or fractional). The contest problems are classified into difficulty levels from 1 (easiest) to 5 (hardest) and span the following topics: number theory, intermediate algebra, algebra, prealgebra, geometry, counting and probability, precalculus. The standard number of in-context examples is 8, following the official dataset.

We provide a flag, `use_chain_of_thought`, which includes the solution when on.

An example input with 0 in-context examples, where `use_chain_of_thought` is off, may be: `Given a mathematics problem, determine the answer. Simplify your answer as much as possible. ### Problem: What is $\ dbinom{n}{n}$ for any positive integer $n$? Answer: $`. The expected output is `1$`.

Data. The data comes from the **MATH** dataset (Hendrycks et al., 2021d).

Pre-processing. We use the same pre-processing code as in the official repository for the dataset, with one exception. Due to the preprocessor implementation, our formatter includes an extra `###` following the instructions and preceding the first in-context learning example.

When the `use_chain_of_thought` setting is turned on, all instances include the solution. The answer is always pre-processed to remove the boxed LaTeX environment, leaving only the value inside LaTeX dollar sign delimiters. The evaluation code matches the official repository implementation as well.

E.3.7 HumanEval

Scenario Description. **HumanEval** (Chen et al., 2021) is a reasoning scenario that features handwritten programming problems. As an example, an input for **HumanEval** looks like: `def return1():\n`, with the desired output: `return 1`. In general, for this scenario, inputs will include a function signature and optionally a docstring, and outputs will contain the body of the function. Accuracy for this scenario is measured using pass@k and correctness rate.

Data. Chen et al. (2021) created the dataset manually with 164 test examples.

Pre-processing. No special pre-processing is performed beyond tokenization.

Data Resources. The dataset can be accessed at `https://www.github.com/openai/human-eval`.

E.3.8 APPS

Scenario Description. **APPS** is a benchmark for measuring the ability of language models trained on source code to complete coding competition style problems(Hendrycks et al., 2021b). The benchmark contains a large collection of Python programming problems. Each instance in the dataset contains a textual description of the coding problem, a list of test cases to verify the correctness of solutions, and (potentially) several reference code solutions. The goal for a machine learning model is then to produce source code given the textual descriptions to pass all given test cases.

Data. The APPS dataset is scraped from various coding challenge websites including AtCoder, CodeChef, Codeforces, Codewars, HackerRank, Kattis, and LeetCode. We refer the reader to their paper for more details(Hendrycks et al., 2021b).

Pre-processing. We mostly follow the same pre-processing steps as in the original paper. To sample solutions, we use a temperature of 0.2, as this was suggested to be the "optimal" temperature for pass@1 evaluation byChen et al. (2021). We heuristically use stop tokens `"'`, `--`, `"""`, and `\n\n\n` terminate generation, and enforce a hard stop when the completion exceeds 600 tokens. To improve performance, we perform in-context learning with 2 randomly sampled training instances from the original training set. To further prevent models from generating code in other programming languages (recall all problems are in Python), we use a prompt of the format `"\nQUESTION:\n" + question + "\n" + starter_code + "\n" + answer_type + "\nANSWER in Python code:\n"`; this last step is slightly different from the construction given by the original authors.

Data Resources. The dataset can be accessed at `https://github.com/hendrycks/apps`.

E.3.9 LegalSupport

Scenario Description. **LegalSupport** is a reasoning scenario that features entailment-style questions grounded in legal language. Each sample consists of a passage extracted from legal text (a "hypothesis"), and two candidate case descriptions, each detailing the a particular court's conclusion regarding a matter of law (the "premises"). The task consists of determining which of the two premises *best* supports the hypothesis. As an example, an input for the scenarios looks like:

> Which statement best supports the passage?
>
> Passage: Rather, we hold the uniform rule is ... that of 'good moral character". Courts have also endorsed using federal, instead of state, standards to interpret federal laws regulating immigration.
>
> Answer A: Interpreting "adultery" for the purpose of eligibility for voluntary departure, and holding that "the appropriate approach is the application of a uniform federal standard."
>
> Answer B: Using state law to define "adultery" in the absence of a federal definition, and suggesting that arguably, Congress intended to defer to the state in which an alien chooses to live for the precise definition ... for it is that particular community which has the greatest interest in its residents moral character.

with the desired output here being "A." We note that **LegalSupport** is intended to capture one type of legal reasoning, but that currently ongoing efforts are exploring legal reasoning in foundation models in greater depth (Guha et al., 2022). The evaluation set is balanced in terms of how many times "A" or "B" is the correct answer.

Accuracy for this scenario is measured using accuracy.

Data. This is a new dataset. This benchmark is constructed from state and federal legal opinions written after 1965. We leverage several properties of legal writing to construct this benchmark. In short–we rely on the fact that lawyers and judges are required to follow a rigid set of rules when citing cases.[104] These rules specify that when citing a case in legal text, the author must both describe how the case supports their argument (in a parenthetical), and state the level of support provided by the case description (via a prefix, known to lawyers as "introductory signals"). As lawyers often support an assertion with multiple citations to different cases–each supporting the assertion to a different extent–we can mine these citations to create our task.

Specifically, we identify sentences in US court opinions for which the author has included two supports with different introductory signals (e.g. one authority is supported with a see signal, while another is supported with a see also signal) and each authority has an associated parenthetical (using the CAP repository).[105] If there are multiple supports, we pick two at random. We extract the assertion, parentheticals, and introductory signals. We treat the assertion as the context, and each parenthetical as a plausible support. We use the definitions of introductory signals to determine which parenthetical best supports the assertion. So for instance, a parenthetical associated with a *see* signal would provide stronger support than a parenthetical associated with a *see also* signal.

The train-dev-test splits for the dataset are 13862/3125/3047 with inputs on average being 137 tokens and outputs on average being 1 token based on the GPT-2 tokenizer.

Pre-processing. No preprocessing is done other than tokenization.

E.3.10 LSAT

Scenario Description. **LSAT** is a reasoning scenario that features analytical reasoning questions from the Law School Admission Test. As an example, an input for the scenarios looks like: "Four students—Fred,

[104] These rules are codified in "The Bluebook", which can be found here: `https://www.legalbluebook.com/`
[105] `https://case.law/`